

Question Bank — Week 1

Why this course · Python basics (Lecture 01, Lab 01) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Answers are in a separate file: [week01-answers.md](#). Attempt every question on paper first. Each question is tagged with its level: **[what]** — recall, **[how]** — apply, **[why]** — explain and justify. Marks shown for written questions are a guide to the length of answer expected.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W1-M01 – W1-M24
B	Short answer and essay	W1-E1 – W1-E6
C	Trace the code — what is printed?	W1-T1 – W1-T6
D	Find and fix the bug	W1-B1 – W1-B3
E	Write the code — checked by pytest	W1-C1 – W1-C4

Part A — Multiple choice

W1-M01 [what] “Every general-purpose programming language is Turing-complete.” What does that tell you?

- **a)** All languages run equally fast
- **b)** Any problem solvable in one of them is solvable in any other
- **c)** All languages differ only in syntax
- **d)** A program written in one language runs unchanged in any other

W1-M02 [what] Python’s type system is best described as:

- **a)** static and strong
- **b)** static and weak
- **c)** dynamic and weak
- **d)** dynamic and strong

W1-M03 [how] What does “3” + 5 do in Python?

- **a)** evaluates to “35”
- **b)** evaluates to 8
- **c)** raises TypeError
- **d)** raises SyntaxError

W1-M04 [what] How does Rust manage memory?

- **a)** The programmer calls `free` for every allocation
- **b)** A garbage collector frees unreachable objects at run time
- **c)** Ownership rules, checked by the compiler, guarantee each allocation is freed exactly once
- **d)** It never frees memory until the program ends

W1-M05 [what] Where and why was C created?

- **a)** By Dennis Ritchie at Bell Labs, to write the Unix operating system
- **b)** By Bjarne Stroustrup at Bell Labs, to add classes to Simula
- **c)** By Donald Knuth, to typeset *The Art of Computer Programming*
- **d)** By Guido van Rossum, as a teaching language

W1-M06 [what] An **Abstract Data Type** specifies:

- **a)** how the data is laid out in memory
- **b)** which programming language the structure must be written in
- **c)** the class hierarchy of the implementation
- **d)** the operations, what they mean and what they cost — nothing about storage

W1-M07 [how] Which of these is the *declarative (comprehension)* style from Lecture 01?

- **a)** `for n in numbers: if n % 2 == 0: total += n * n`
- **b)** `total = sum(n * n for n in numbers if n % 2 == 0)`
- **c)** `total = sum(map(lambda n: n * n, filter(lambda n: n % 2 == 0, numbers)))`
- **d)** `while i < len(numbers): ...`

W1-M08 [what] What does the TIOBE index actually count?

- **a)** search-engine results for “X programming”
- **b)** the quality of each language
- **c)** commits on GitHub
- **d)** self-reported usage in a developer survey

W1-M09 [why] In the course’s three levels of knowledge, which is **know-why**?

- **a)** You can define a stack
- **b)** You can implement a stack that passes its tests
- **c)** You can say why a stack and not a queue, and what it costs
- **d)** You can recite the Big-O of every stack operation

W1-M10 [what] To pass this course, a student needs:

- **a)** at least 50% overall
- **b)** at least 60% overall and at least 30% of the final exam
- **c)** at least 60% overall, whatever the final exam mark
- **d)** at least 30% of the final exam, whatever the overall mark

W1-M11 [how] What is `-7 // 2`?

- **a)** -3
- **b)** -3.5

- **c)** 3
- **d)** -4

W1-M12 [how] What is `-7 % 3`?

- **a)** 2
- **b)** -1
- **c)** 1
- **d)** -2

W1-M13 [how] What is `2 ** 3 ** 2`?

- **a)** 64
- **b)** 512
- **c)** 36
- **d)** 12

W1-M14 [how] What is `type(8 / 4)`?

- **a)** `<class 'int'>`
- **b)** `<class 'str'>`
- **c)** `<class 'float'>`
- **d)** it raises `TypeError`

W1-M15 [how] With `word = "Python"`, what is `word[1:4]`?

- **a)** "Pyt"
- **b)** "ytho"
- **c)** "ython"
- **d)** "yth"

W1-M16 [how] What is `"Mansoura"[::-2]`?

- **a)** "ausa"
- **b)** "aruosnaM"
- **c)** "Mnor"
- **d)** "arsa"

W1-M17 [how] After `a = [1, 2, 3]`, `b = a`, `b.append(4)`, what is `a`?

- **a)** `[1, 2, 3]`
- **b)** `[4, 1, 2, 3]`
- **c)** `[1, 2, 3, 4]`
- **d)** `[[1, 2, 3], 4]`

W1-M18 [how] After `a = [1, 2]` and `c = a[:]`, what are `c is a` and `c == a`?

- **a)** `True, True`
- **b)** `False, True`
- **c)** `True, False`
- **d)** `False, False`

W1-M19 [how] `grid = [[0] * 2] * 2`, then `grid[0][0] = 1`. What is `grid`?

- **a)** `[[1, 0], [0, 0]]`
- **b)** `[[1, 1], [0, 0]]`
- **c)** `[[1, 0], [1, 0]]`
- **d)** `[[1, 1], [1, 1]]`

W1-M20 [what] What is `round(2.5) + round(3.5)`?

- **a)** 6
- **b)** 7
- **c)** 5
- **d)** 8

W1-M21 [why] Why is `0.1 + 0.2 == 0.3` False in Python?

- **a)** Python has a bug in float addition
- **b)** `0.1` is stored as an `int`
- **c)** `==` compares identity, not value, for floats
- **d)** `0.1`, `0.2` and `0.3` cannot be stored exactly in binary floating point

W1-M22 [how] What happens when you run `s = "abc"` then `s[0] = "z"`?

- **a)** `s` becomes `"zbc"`
- **b)** a `TypeError` is raised, because strings are immutable
- **c)** a new string `"zbc"` is created and `s` refers to it
- **d)** an `IndexError` is raised

W1-M23 [how] A function's body is `print(width * height)` and nothing else. What does `x = area(3, 4)` store in `x`?

- **a)** 12
- **b)** `"12"`
- **c)** None
- **d)** nothing — it is a `NameError`

W1-M24 [how] What does `int("4.5")` do?

- **a)** returns 4
- **b)** returns 5
- **c)** returns 4.5
- **d)** raises `ValueError`

Part B — Short answer and essay

W1-E1 [why] (4 marks) "All programming languages are the same; only the syntax differs." Explain what is true in this statement and what is wrong with it. Give one concrete example of a design decision that makes two languages genuinely different.

W1-E2 [why] (4 marks) Explain the two independent axes of type systems. Place Python and JavaScript on both axes, and show with one expression how their behaviour differs.

W1-E3 [what] (3 marks) Describe the three approaches to memory management from Lecture 01, naming one language that uses each. Which one does Python use, and why does that make data structures harder to see in Python?

W1-E4 [why] (4 marks) Distinguish an **Abstract Data Type** from a **data structure**. Illustrate with the Stack ADT and two different structures that implement it.

W1-E5 [why] (3 marks) Give two reasons the course uses Python, and the honest drawback of Python *for this course*. How does the course deal with that drawback?

W1-E6 [why] (3 marks) "In Python a variable is not a box holding a value; it is a name bound to an object." Explain this with a short code example and a diagram, and state one consequence for lists.

Part C — Trace the code

For each program, write **exactly** what is printed.

W1-T1 [how]

```
print(17 // 5, 17 % 5, -17 // 5, -17 % 5)
print(divmod(23, 4), 2 ** 10, 7 / 2)
```

W1-T2 [how]

```
s = "Data Structures"
print(s[0], s[-1], s[5:9], s[:4].upper())
print(s.split(), len(s), s.count("t"))
```

W1-T3 [how]

```
a = [1, 2, 3]
b = a
c = a[:]
a.append(4)
b[0] = 99
c = c + [5]
print(a, b, c)
a += [6]
print(b)
```

W1-T4 [how]

```
n, count = 1000, 0
while n > 1:
    n = n // 3
    count += 1
print(n, count)
```

W1-T5 [how]

```
name, mark = "Omar", 87.456
print(f"{name:>6}|{mark:.1f}|{mark:8.2f}|{len(name):03d}")
```

W1-T6 [how]

```
a, b = 0, 1
result = []
while len(result) < 8:
    result.append(a)
    a, b = b, a + b
print(result)
print(result[-3:], result[::-3])
```

Part D — Find and fix the bug

Each function has **one** bug. Say what goes wrong, for which input, and fix it.

W1-B1 [how] Should return the initials of a name, e.g. "H.E.".

```
def initials(full_name):
    result = ""
    for word in full_name.split(" "):
        result += word[0].upper() + "."
    return result
```

W1-B2 [how] Should return the average of a non-empty list.

```
def average(values):
    total = 0
    i = 0
    while i < len(values):
        total += values[i]
    return total / len(values)
```

W1-B3 [how] Should return a copy of values with item added at the end, **without changing** values.

```
def with_item(values, item):
    result = values
    result.append(item)
    return result
```

Part E — Write the code

Write these in `practice/week01.py` and check with `pytest tests/test_practice_week01.py -v`. The docstrings in that file are the full specification.

W1-C1 [how] `count_vowels(text)` — how many of a, e, i, o, u appear in text, in either case. `count_vowels("Data Structures")` is 5.

W1-C2 [how] `second_largest(values)` — the largest value strictly smaller than the maximum, in **one pass** and **without sorting**. Raise `ValueError` if there is no such value. `second_largest([4, 9, 2, 9, 7])` is 7.

W1-C3 [how] `compress(text)` — run-length encoding: `compress("aaabcc")` is "a3b1c2".

W1-C4 [how] `is_anagram(first, second)` — same letters, same counts, ignoring case and spaces. `is_anagram("Dormitory", "dirty room")` is True.